

PromptToolkit

Comparador de Técnicas de Prompting

Checkpoint 02 – Prompt Engineering & Artificial Intelligence

FIAP – 2026

Domínio: E-commerce / Atendimento ao Cliente

Integrantes do Grupo:

Davi Simoncelo – RM571738

Augusto de Souza Ávila – RM:570839

Murilo Lima de Carvalho - RM:570156

João Pedro Sousa - RM:573962

Matheus Evangelista - RM:568593

1. Domínio e Problema

Nós escolhemos o domínio de E-commerce e Atendimento ao Cliente porque é uma área que gera uma quantidade enorme de texto todo dia. Lojas como Mercado Livre, Amazon e Magazine Luiza recebem milhares de reviews, reclamações e mensagens de clientes, e analisar tudo isso manualmente é inviável.

A ideia do toolkit é justamente automatizar esse trabalho. Em vez de alguém ficar lendo review por review, o sistema manda o texto pra uma LLM e ela faz a análise. Mas a grande questão é: qual técnica de prompting funciona melhor pra cada tipo de tarefa? É isso que o toolkit responde.

Tarefas que nós testamos:

- Classificação de Sentimento - o modelo recebe uma review e tem que dizer se é POSITIVO, NEGATIVO, NEUTRO ou MISTO
- Extração de Dados de Pedido - recebe uma reclamação e extrai produto, preço, problema e número do pedido
- Sumarização de Reviews - recebe uma review longa e resume em 2-3 frases

2. Arquitetura

O projeto é modular, cada arquivo faz uma coisa. O fluxo geral funciona assim:

`inputs.json` → `prompt_builder` → `techniques` → `llm_client` → `evaluator` → `report` → `output/`

Passo a passo do que acontece quando roda o `main.py`:

1. Carrega as configurações do `.env` (host do Ollama, modelo, timeout)
2. Pra cada tarefa definida no `tasks.py`, puxa os inputs reais do `inputs.json`
3. Aplica as 4 técnicas (Zero-Shot, Few-Shot, CoT, Role) usando o `techniques.py`
4. Cada técnica monta o prompt de um jeito diferente (`prompt_builder.py`)
5. Manda pro Ollama via API REST (`llm_client.py`)
6. Compara a resposta com o esperado e calcula acurácia (`evaluator.py`)
7. No final, gera CSV, gráficos e recomendação (`report.py`)
8. Ainda testa 3 temperaturas (0.1, 0.5, 1.0) no melhor prompt de cada tarefa

Estrutura de Pastas

```
prompt-toolkit/
├── main.py
├── src/
│   ├── llm_client.py
│   ├── prompt_builder.py
│   ├── techniques.py
│   ├── tasks.py
│   ├── evaluator.py
│   └── report.py
├── data/
│   ├── inputs.json
│   └── examples.json
├── prompts/
│   ├── system_prompts.json
│   └── templates.json
└── output/
    ├── resultados.csv
    └── graficos/
```

3. Stack Técnica

Tudo roda local, sem precisar de API paga. As ferramentas que usamos:

- Python 3.10+
- Ollama - roda a LLM localmente (modelo gpt-oss)
- tiktoken - conta os tokens do prompt (encoding cl100k_base)
- matplotlib - gera os gráficos de barras
- pandas - organiza os dados em tabela
- requests - faz as chamadas HTTP pro Ollama
- python-dotenv - carrega as variáveis do .env
- tabulate - formata tabelas no terminal

Como rodar:

```
pip install -r requirements.txt
cp .env.example .env
ollama serve          # em outro terminal
ollama pull gpt-oss
python main.py
```

4. Resultados

Essa é a tabela com os resultados que o toolkit gerou:

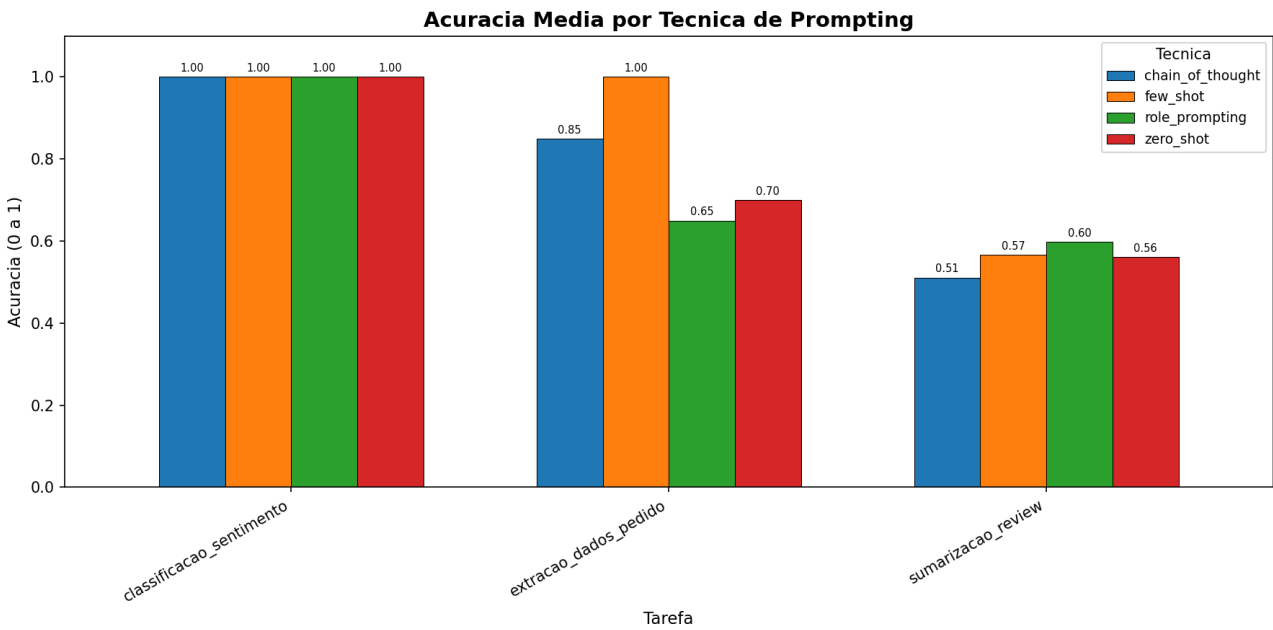
Tarefa	Técnica	Acurácia	Tokens	Tempo(ms)
Classif. Sentimento	zero_shot	100,0%	177	4636
Classif. Sentimento	few_shot	100,0%	261	2692
Classif. Sentimento	chain_of_thought	100,0%	272	3254
Classif. Sentimento	role_prompting	100,0%	305	2518
Extração Dados	zero_shot	70,0%	173	6517
Extração Dados	few_shot	100,0%	337	6292
Extração Dados	chain_of_thought	85,0%	266	8263
Extração Dados	role_prompting	65,0%	309	7511
Sumarização	zero_shot	56,1%	250	7179
Sumarização	few_shot	56,7%	567	9459
Sumarização	chain_of_thought	51,1%	334	8819
Sumarização	role_prompting	59,8%	378	8397

O que o toolkit recomendou:

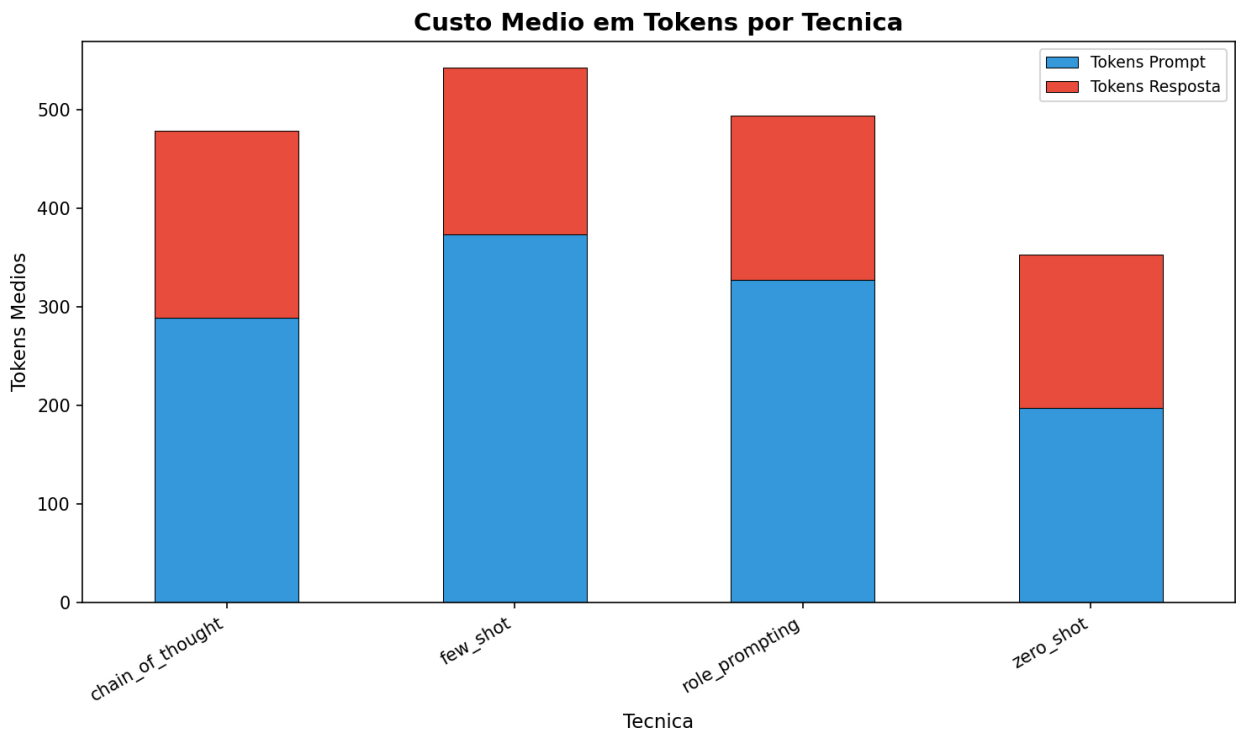
- Classificação de Sentimento → zero_shot ganhou. Todas as técnicas acertaram 100%, mas o zero-shot usa menos tokens (177), então é o mais barato.
- Extração de Dados → few_shot ganhou (100%). Os exemplos ajudam o modelo a entender o formato de saída esperado.
- Sumarização → role_prompting ganhou (59,8%). A persona de analista CX ajuda o modelo a focar nos pontos importantes da review.

5. Gráficos

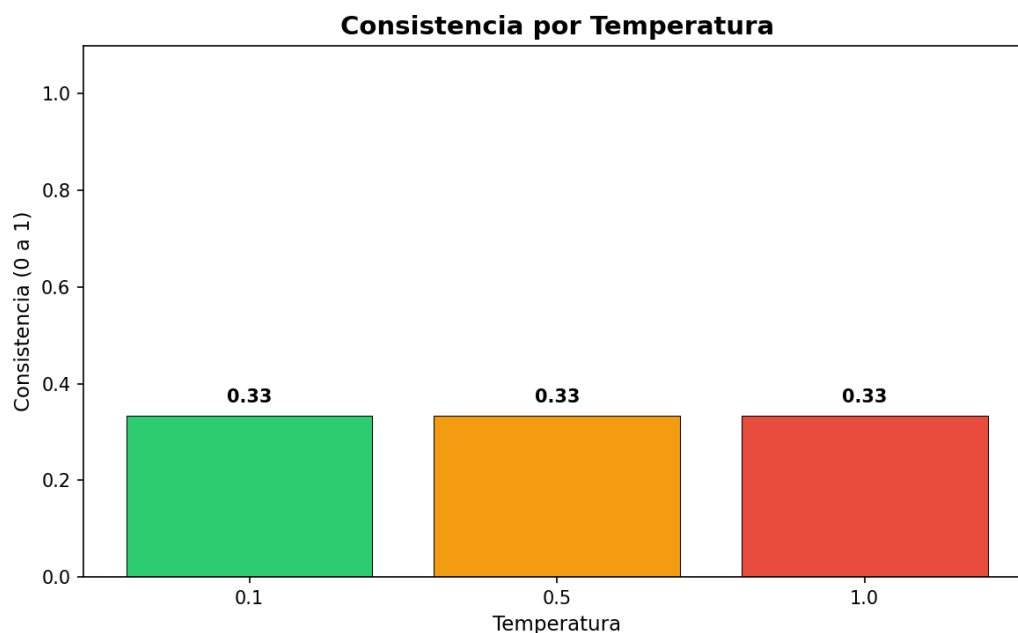
Acurácia por Técnica



Custo em Tokens por Técnica



Consistência por Temperatura



6. Guia de Bolso

Zero-Shot

Usar quando a tarefa é simples e direta, tipo classificar sentimento. Não precisa de exemplo nenhum, é só mandar a instrução. É o mais barato em tokens e o mais rápido de montar.

Few-Shot

Usar quando o formato da resposta importa muito, tipo extrair dados em JSON. Você coloca 2-3 exemplos no prompt e o modelo entende o padrão. Gasta mais tokens por causa dos exemplos, mas a precisão compensa.

Chain-of-Thought

Usar quando precisa de raciocínio, tipo analisar algo complexo. O modelo pensa passo a passo antes de dar a resposta final. As respostas ficam maiores, mas dá para acompanhar a lógica.

Role Prompting

Usar quando precisa de uma perspectiva específica, tipo sumarizar com visão de negócio. Você define uma persona no system prompt e o modelo adota aquele papel. Funciona bem para tarefas que precisam de contexto especializado.

7. Reflexão

O que surpreendeu?

O que mais surpreendeu foi o zero-shot ter dado 100% na classificação de sentimento. A gente achava que ia precisar de exemplos ou raciocínio passo a passo pra acertar, mas uma instrução simples e clara já foi suficiente. Isso mostra que nem sempre a técnica mais complexa é a melhor – às vezes o simples resolve.

Qual técnica venceu?

Não teve uma técnica que ganhou em tudo. Cada tarefa teve a sua melhor: o zero-shot para a classificação (mais barato e igual de preciso), o few-shot pra extração (os exemplos ensinam o formato), e o role prompting para a sumarização (a persona de analista CX ajuda a focar no importante). A conclusão é que a escolha da técnica depende do tipo de tarefa.

O que a gente faria diferente?

Se tivesse mais tempo, iríamos testar combinar técnicas (tipo Role + Few-Shot juntos), colocar mais inputs pra ter uma amostra maior, e usar métricas mais sofisticadas para a sumarização (tipo ROUGE score ao invés de comparar palavras-chave). Também seria legal testar com modelos de tamanhos diferentes para ver se um modelo menor consegue resultados parecidos.